HILBERT: A Wideband Quadrature Transformer

The Mathematics of the SEAM-LTM Hilbert Plugin

Giuseppe Silvi — SEAM

2026

Abstract

This document derives the mathematics of the `hilbert` plugin step by step. It develops the two all-pass topologies the plugin ships, the RBJ biquad cascade and the polyphase first-order pair, and it explains the quadrature property they share. It then documents the topologies the plugin sets aside, the elliptic half-band form and the FIR analytic-signal form, so that a reader can study the differences that motivated the choices. The FIR discussion analyses Miller Puckette's Pure Data `hilbert~` object, which serves as the bridge between the phase-difference family the plugin implements and the analytic-signal form it documents.

Contents

1 Introduction and Pedagogical Framing

A quadrature transformer takes one real input and produces two outputs whose phase differs by 90° across the audio band. The `hilbert` plugin realises this as a mono input mapped to an in-phase output and a quadrature output. The plugin belongs to the SEAM-LTM suite, whose purpose is to teach digital signal processing by reading clear C++ source alongside its mathematical specification.

The suite documents the choices behind each plugin and also the alternatives it sets aside. Documenting a non-choice records the characteristic that made another path preferable, and

it prepares study material for a reader who wants to understand the trade-off. The `hilbert` plugin ships two topologies, the RBJ biquad cascade and the polyphase first-order pair, and this document develops both. It then documents two non-choices, the elliptic half-band form and the FIR analytic-signal form, and it explains what each one offers and what it costs.

2 The Quadrature Property

The plugin forms two filtered versions of the input, $y_R = H_R\{x\}$ on the in-phase path and $y_I = H_I\{x\}$ on the quadrature path. Both H_R and H_I are all-pass networks, so each output preserves the magnitude spectrum of the input and changes only its phase. The design constrains the difference of the two phase responses to hold $-90°$ across 20 Hz to 20 000 Hz:

$$\varphi_{H_I}(\omega) - \varphi_{H_R}(\omega) = -\frac{\pi}{2}, \qquad \omega \in [\omega_{\mathrm{lo}}, \omega_{\mathrm{hi}}]. \tag{1}$$

The quadrature relationship is a property of the pair, because the design fixes the relative phase of the two branches and leaves the absolute phase of each branch free. Both outputs therefore differ from the dry input, and the in-phase output is itself an all-pass-filtered signal rather than the original. This structural fact distinguishes the phase-difference family, which the plugin implements, from the analytic-signal form of Section 7, which returns the delayed input on its real branch.

3 RBJ Topology: A Matched Biquad Cascade

The first topology realises each path as a cascade of second-order all-pass sections parameterised by a centre frequency and a quality factor. The analog second-order all-pass prototype and its phase response are:

$$H_a(s) = \frac{s^2 - (\omega_0/Q)\, s + \omega_0^2}{s^2 + (\omega_0/Q)\, s + \omega_0^2}, \qquad \omega_0 = 2\pi f_0, \tag{2}$$

$$\varphi_a(\omega) = -2 \arctan \frac{(\omega_0/Q)\, \omega}{\omega_0^2 - \omega^2}. \tag{3}$$

The in-phase path H_R and the quadrature path H_I each use three sections. A phase fit optimises the six (f_0, Q) pairs so that $\varphi_{H_I} - \varphi_{H_R} = -90°$ over the audio band [1, 2]. The RBJ bilinear realisation maps each analog section to a digital biquad all-pass [3]:

$$a_1 = \frac{-2\cos\omega}{1+\alpha}, \quad a_2 = \frac{1-\alpha}{1+\alpha}, \quad \alpha = \frac{\sin\omega}{2Q}, \quad \omega = \frac{2\pi f_0}{f_s}, \tag{4}$$

with all-pass numerator $b = [a_2,\ a_1,\ 1]$.

The bilinear transform maps a digital frequency f to the analog frequency $F = (f_s/\pi)\tan(\pi f/f_s)$. A fixed (f_0, Q) set therefore presents its broadband phase on a warped axis that depends on f_s, and the quadrature holds near the sample rate the set was designed for. Sample-rate independence belongs to the design procedure: the plugin re-runs the phase fit at the host sample rate, which restores the quadrature at that rate. This is the same live-design engine that the `x2uhj` encoder uses for its UHJ quadrature pair, and at 48 kHz the two share the same coefficients. The cascade costs four multiplies per section, so the matched pair of six sections costs 24 multiplies per sample.

4 Polyphase Topology: A First-Order Pair

The second topology realises each path as a cascade of first-order all-pass sections, with a one-sample relative delay on the quadrature path. Each section has a single coefficient a and one multiply per sample:

$$H_{\mathrm{ap}}(z) = \frac{a^2 - z^{-2}}{1 - a^2 z^{-2}}. \tag{5}$$

The two paths H_A and H_B use four sections each, and the quadrature output applies a one-sample delay z^{-1} to path B:

$$y_R = H_A\{x\}, \qquad y_I = z^{-1} H_B\{x\}. \tag{6}$$

This polyphase pair is the classic audio realisation of the broadband 90° difference [4, 5, 6]. The plugin seeds the eight coefficients with Olli Niemitalo's published order-four set [6] and re-optimises them at the host sample rate with the same phase fit used for the RBJ cascade. The structure rests on half-band and Hilbert transformer theory [7], and the normalised-frequency design tradition reaches back to the analog phase-difference networks [8]. The eight first-order sections cost 8 multiplies per sample, which is one third of the RBJ cost.

5 Cost and Accuracy

Table 1 states the trade-off the two topologies present at 48 kHz. The RBJ cascade reaches the smaller phase error, and the polyphase pair reaches a comparable error at one third of the multiply count. This contrast is the pedagogical payload of the plugin: a listener compares accuracy against cost on the same source by switching the topology live.

Table 1: Shipped topologies at 48 kHz. Order is sections per branch; cost is multiplies per sample for the matched pair.

Topology	Order	Max error @48k	Cost (mult/sample)
RBJ biquad cascade	3	1.36°	24
Polyphase first-order	4	1.67°	8

The live-design engine holds these errors across the standard rates. The in-plugin readout reports the achieved error at the host sample rate, and a goniometer confirms the relationship visually: a single tone through the I/Q pair traces a circle, because two sinusoids in constant −90° relation map the in-phase and quadrature outputs onto orthogonal axes.

6 Non-Choice: The Elliptic Half-Band Form

The elliptic half-band topology realises the quadrature pair through a half-band filter whose polyphase components are all-pass networks. Its Cauer synthesis places the poles and zeros by Jacobi elliptic functions, which produces an equiripple phase difference with few sections [7]. The synthesis is a self-contained numerical module, and its coefficients follow from a closed elliptic procedure that is independent of the sample rate. This independence places the elliptic form outside the live-design shape of the present iteration, which re-optimises every topology at the host rate, so the plugin documents it here and sets its implementation aside for a dedicated future iteration.

7 Non-Choice: The FIR Analytic-Signal Form

The ideal Hilbert transform $\hat{x}(t)$ shifts every frequency component of $x(t)$ by $-90°$, and it forms the analytic signal $x_a(t) = x(t) + j\,\hat{x}(t)$ whose spectrum vanishes for negative frequencies [9, 10]. The analytic signal supplies an instantaneous amplitude and an instantaneous phase, which makes it the textbook object behind single-sideband modulation, envelope following, and frequency shifting.

A finite impulse response realises the transform with a fixed tap set and a known group delay. The real branch delays the input by that group delay, and the imaginary branch applies the FIR Hilbert kernel, so the real output equals the delayed input and the pair realises the analytic signal directly. The realisation fixes the kernel to one length, and its accuracy at the band edges follows from the window and the tap count, so it carries a latency that the all-pass forms avoid.

7.1 Puckette's `hilbert~` as the Bridge

Miller Puckette's Pure Data ships a `hilbert~` object, and its patch builds each output from two `biquad~` sections in series, so each branch is a fourth-order all-pass and the pair holds about 90° between the branches [11]. The patch adapts a 4X design by Emmanuel Favreau from around 1982, and its coefficients are fixed. Both branches are all-pass filtered and neither branch equals the input, so the object belongs to the phase-difference family that the `hilbert` plugin implements.

The Pure Data `biquad~` is a canonical direct-form-II section with the difference equations

$$w[n] = x[n] + fb_1\, w[n-1] + fb_2\, w[n-2], \qquad y[n] = ff_1\, w[n] + ff_2\, w[n-1] + ff_3\, w[n-2], \quad (7)$$

which realises $H(z) = (ff_1 + ff_2 z^{-1} + ff_3 z^{-2})/(1 - fb_1 z^{-1} - fb_2 z^{-2})$. A section is all-pass when its numerator is the denominator reversed, that is $(ff_1, ff_2, ff_3) = (-fb_2, -fb_1, 1)$, and every `biquad~` in `hilbert~` satisfies this condition exactly.

Each of the four sections has real poles, so each factors into two first-order real all-pass sections of the form $(z^{-1} - a)/(1 - az^{-1})$. This first-order section is the all-pass primitive that the Pure Data examples build from a real one-pole `rpole~`, namely $1/(1 - az^{-1})$, in series with a real reversed one-zero `rzero_rev~`, namely $z^{-1} - a$, which places a pole at a and a zero at its reciprocal $1/a$. Table 2 lists the eight coefficients that result from factoring the patch. Puckette's transformer is therefore a cascade of first-order real all-pass sections, the same family as the plugin's polyphase topology, with coefficients fixed at the design's original rate, so its quadrature drifts as the sample rate departs from that rate.

Table 2: Pure Data `hilbert~` factored into first-order real all-pass sections. Each branch is two `biquad~` in series, and each `biquad~` factors into two sections $(z^{-1} - a)/(1 - az^{-1})$.

Branch	First-order all-pass coefficient a
Branch 1	0.99485
Branch 1	0.95147
Branch 1	0.75364
Branch 1	0.08410
Branch 2	0.49771
Branch 2	−0.52340
Branch 2	0.98048
Branch 2	0.88802

The contrast between Puckette's object and the FIR analytic-signal form makes the structural point concrete. The phase-difference network constrains only the relative phase of the

two branches, so it returns two filtered signals and holds the quadrature with low latency. The analytic-signal FIR constrains the absolute phase of each branch, so it returns the delayed input on the real branch and realises $x_a(t)$ at the cost of latency. A reader who hears both forms on the same source hears why the all-pass pair never returns the dry signal, which is the distinction this section documents.

7.2 Why the Plugin Documents Rather Than Ships the FIR Form

The FIR form sits outside the suite's Faust specification path. The SEAM Faust libraries express the all-pass topologies as cascades of standard biquad and first-order sections, which the C++ plugin re-implements by hand. A FIR Hilbert kernel needs a convolution against an externally designed tap set, which calls for a new on-demand Faust primitive, so the Faust specification carries the all-pass pair and this document carries the FIR analysis. The plugin therefore documents the analytic-signal form for study and keeps the shipped topologies within the live-design engine.

8 C++ Core

The following listing is the runtime section that both topologies share: a first-order polyphase all-pass realised as a biquad in z^{-2}, with the coefficients of $H_{\text{ap}}(z) = (a^2 - z^{-2})/(1 - a^2 z^{-2})$.

```
    // Polyphase first-order all-pass: H(z) = (a^2 - z^-2)/(1 - a^2 z^-2).
    static Biquad makePolyphaseBiquad(double a) {
        const double a2 = a * a;
        Biquad bq; bq.b0 = a2; bq.b1 = 0.0; bq.b2 = -1.0;
                   bq.a1 = 0.0; bq.a2 = -a2;
        return bq;
    }
    // x -> (inPhase, quad); quad lags inPhase by 90 degrees across the band.
    inline void process(double x, double& inPhase, double& quad) {
        double a = x; for (int i = 0; i < nA_; ++i) a = pathA_[i].process(a);
        double b = x; for (int i = 0; i < nB_; ++i) b = pathB_[i].process(b);
        inPhase = a;
        if (mode_ == Mode::Polyphase) { quad = delayB_; delayB_ = b; }
        else                          { quad = b; }
    }
```

9 Realization Topologies

A transfer function $H(z)$ fixes the response, and the structure that computes it is a separate choice. The same second-order all-pass admits several realizations that differ in memory layout and numerical behaviour while they share the response. This document has shown three of them on the same section. Pure Data's `biquad~` is canonical direct form II, which carries a single two-sample state w shared between the recursive and the feed-forward parts. The SEAM C++ core of the previous section is direct form I, which carries separate input and output histories x_1, x_2, y_1, y_2. Faust's `filters.lib` expresses both the direct form and its transpose in a few lines [12], and `seam.filters.lib` builds the quadrature pair on the direct form `fi.tf2`:

```
// Faust filters.lib (Julius O. Smith III): one biquad, two structures.
// Direct form (canonical) -- used by seam.filters.lib aprbj / appoly:
iir(bv,av) = ma.sub ~ fir(av) : fir(bv);
tf2(b0,b1,b2,a1,a2) = iir((b0,b1,b2),(a1,a2));

// Direct form 2 transposed -- identical response, adders on the output:
```

```
tf22t(b0,b1,b2,a1,a2) =
    _ : (_,_,(_ <: *(b2)',*(b1)',*(b0))
      : _,+',_,_ :> _)~*(-a1)~*(-a2) : _;
```

The direct form and the transposed form return the same samples for the same coefficients. The transposed structure reorders the additions onto the output path, which changes the rounding behaviour and the state update while it holds the response. Reading the Pure Data section, the SEAM C++ section, and these Faust definitions side by side shows one all-pass response across direct form I, direct form II, and the transposed direct form II, which separates the mathematical specification from its realization.

A Coefficient Tables

Table 3 lists the RBJ (f_0, Q) pairs the live engine produces at 48 kHz, and Table 4 lists the polyphase coefficients at the same rate.

Table 3: RBJ (f_0, Q) pairs at 48 kHz. Maximum phase error 1.36°.

Path	f_0 (Hz)	Q
H_R	141.887	0.2019
H_R	671.746	0.2122
H_R	18654.238	0.3031
H_I	24.004	0.3090
H_I	2991.937	0.3848
H_I	3219.986	0.0963

Table 4: Polyphase coefficients a at 48 kHz. Maximum phase error 1.67°.

Path	a
A	0.397260394
A	0.853250636
A	0.971682572
A	0.995211876
B	0.687396697
B	0.934674128
B	0.987987539
B	0.998735492

References

[1] M. Lang, "Allpass filter design and applications," *IEEE Transactions on Signal Processing*, vol. 46, no. 9, pp. 2505–2514, 1998.

[2] D. Harris, E. Berdahl, and J. S. Abel, "An infinite impulse response (IIR) hilbert transformer filter design technique for audio," in *Proc. 129th Audio Engineering Society Convention*, 2010, paper 8258.

[3] R. Bristow-Johnson, "Cookbook formulae for audio EQ biquad filter coefficients," Audio EQ Cookbook.

[4] R. Ansari, "IIR discrete-time hilbert transformers," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 35, no. 8, pp. 1116–1119, 1987.

[5] P. A. Regalia, S. K. Mitra, and P. P. Vaidyanathan, "The digital all-pass filter: A versatile signal processing building block," *Proceedings of the IEEE*, vol. 76, no. 1, pp. 19–37, 1988.

[6] O. Niemitalo, "90 degree phase difference IIR allpass pair," https://yehar.com/blog/?p=368, 2003.

[7] H. W. Schüßler and P. Steffen, "Halfband filters and hilbert transformers," *Circuits, Systems and Signal Processing*, vol. 17, no. 2, pp. 137–164, 1998.

[8] S. D. Bedrosian, "Normalized design of 90-degree phase-difference networks," *IRE Transactions on Circuit Theory*, vol. 7, no. 2, pp. 128–136, 1960.

[9] R. N. Bracewell, *The Fourier Transform and Its Applications*, 3rd ed. McGraw-Hill, 2000.

[10] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Prentice Hall, 2010.

[11] M. Puckette, *The Theory and Technique of Electronic Music*. World Scientific, 2007, see also the Pure Data `hilbert~` object and all-pass example patches.

[12] J. O. Smith, "Four direct forms," https://ccrma.stanford.edu/~jos/filters/Four_Direct_Forms.html, online book: Introduction to Digital Filters, CCRMA.